

Advanced Model Selection Workshop

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain why accuracy alone cannot determine the best machine learning model.
- Define bias and variance, and identify underfitting versus overfitting from their combination.
- Apply the bias-variance tradeoff to find a model complexity that generalizes well.
- Compare two classifiers on the iris dataset using scikit-learn tools, and describe how ensemble learning combines multiple models.

2. Detailed Explanation

Why accuracy alone is not enough

Model selection is the process of choosing the best machine learning model for a task, and it is not simply a matter of picking the one with the highest accuracy. A model that reports 99% accuracy is not automatically the best choice, because that number alone does not reveal how the model behaves on unseen data.

A model can score very well on the data it was trained on while performing poorly on new, unseen data. This mismatch is called overfitting: strong training performance paired with weak test performance.

Picture a student who receives 50 exam questions along with their answers, but with no explanation of the underlying concepts. The student memorizes each question-answer pair. In the real exam, unfamiliar questions on the same topics appear, and because the student memorized answers instead of learning the patterns, they cannot solve the new

questions. A model behaves the same way when it memorizes training data instead of learning the patterns inside it: it performs well on data it has already seen and poorly on data it has not. Model selection therefore weighs multiple signals together, never accuracy by itself.

Parameters to consider when selecting a model

Model selection should weigh several factors together, not accuracy in isolation.

Selecting a model well means looking at all of these together:

- Training performance (necessary but not sufficient on its own)
- Validation performance, also called test performance
- Accuracy and F1 score as measures of performance
- Speed
- Hyperparameters
- Bias and variance
- Generalization
- Computational cost, since a model must be practically usable in an industrial setting
- Business requirements

The target is a model that generalizes well, rather than one that only performs well on training data.

Bias

Bias is the error caused by a model being too simple to understand the underlying pattern in the data. When the true relationship between the inputs and the output is complex but the model used is very simple, the model cannot capture that complexity, producing high bias.

A student who has only studied one simple version of a topic will struggle when a complicated question on that same topic appears in the exam. The student learned the relationship at a surface level, so a more complex version of the problem becomes unsolvable. This mirrors high bias: a model trained on an oversimplified view of a relationship fails when the real relationship is more complex.

House price prediction is a concrete example. Price can depend on many factors: area, location, number of bedrooms (for example, 3BHK or 4BHK), the age of the building, parking, and amenities such as a gated society. A model that only learns a rule like "price equals area times a constant," or "price equals location rank times a constant," ignores every other factor. This oversimplified model is another example of high bias, because it cannot capture the true, more complex relationship between all the relevant factors and price.

Variance

Variance describes how much a model's predictions change when the training data changes. In everyday language, variance means fluctuation. A model with high variance is the opposite of robust: small changes to the training data cause large changes in the model's predictions.

High variance means the model is learning the training data very tightly, almost memorizing it, and learning the data this tightly leads to overfitting.

Watch out for: a very high training accuracy is not automatically good news. If it comes with poor test performance, it is a sign of high variance and overfitting, not a sign the model is ready to ship.

How bias and variance combine: underfitting and overfitting

The two errors trade off in a predictable pattern:

Condition	Bias	Variance	Result
Model too simple	High	Low	Underfitting
Model too complex	Low	High	Overfitting
Balanced model	Moderate	Moderate	Good generalization

A model that is too simple has high bias and low variance, because a simple model does not memorize the data — this leads to underfitting. Underfitting means the model is unable to learn the important patterns in the data, so training performance is poor and test performance is poor too, since the model never learned the patterns in the first place.

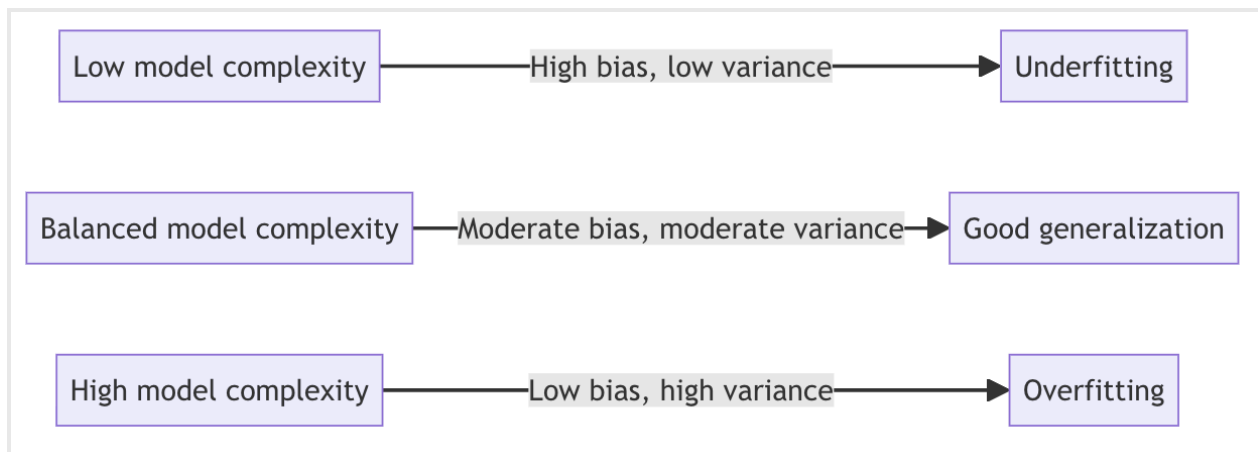
A model that is too complex has low bias, because its error on training data is small, but high variance, because it overfits. Overfitting occurs when a model learns the training data too closely, which leads to poor generalization. The clearest sign of overfitting is very good training accuracy combined with poor test performance.

Neither extreme is desirable. A good fit model shows strong training performance and comparably strong test performance — for example, high training accuracy alongside roughly 92% test accuracy, indicating that it generalizes well rather than memorizing the training set. Training performance measures how well a model has learned the data it was trained on; validation performance measures how well the model is likely to perform on unseen data.

The bias-variance tradeoff

A tradeoff means increasing something at the cost of decreasing something else — like improving health at the cost of giving up junk food. In modeling terms, the bias-variance tradeoff describes the relationship between error and model complexity.

Plotting error on one axis and model complexity on the other, the goal is to find the complexity level at which error is lowest. As model complexity increases from a very low starting point, bias decreases. When complexity keeps increasing and reaches a very high level, the model overfits and variance becomes high. Decreasing bias by increasing complexity comes at the cost of increasing variance. The bias-variance tradeoff is the search for the complexity level that balances the two, rather than optimizing either one in isolation.



Ensemble learning

Ensemble learning combines multiple models to produce a stronger overall prediction, rather than relying on a single model's output.

When making an important life decision, such as choosing a career, relying on the opinion of a single person from a single profession is risky. Consulting multiple people across different professions produces a more balanced view, because no single

person's bias dominates the decision. Ensemble models apply the same idea: instead of trusting one model, multiple models are combined so that no single model's errors dominate the final prediction.

The method used to combine multiple models' outputs depends on the type of data:

- For numerical data, the outputs are combined using averaging.
- For categorical data, the outputs are combined using majority voting, also called the mode. Mode means majority: whichever class the most models predict becomes the ensemble's final prediction.

Tools for a bias-variance demonstration: linspace and pipelines

Before building a hands-on demo, two supporting tools were introduced.

`np.linspace` generates evenly spaced numbers, including both endpoints:

```
import numpy as np

a = np.linspace(0, 10, 6)
# a = [0, 2, 4, 6, 8, 10]
# 6 equally spaced numbers between 0 and 10, both endpoints included

b = np.linspace(0, 10, 5)
# b = [0, 2.5, 5, 7.5, 10]
# 5 equally spaced numbers between 0 and 10, both endpoints included
```

`linspace` always returns numbers in float form. Unlike some other range-generating tools, both the start and the end values are included in the output.

For a planned bias-variance demonstration using sample data, the intended tools were `mean_squared_error` from `sklearn.metrics`, polynomial features, and a simple linear regression model, wired together with a pipeline:

```
from sklearn.metrics import mean_squared_error
```

A pipeline lets multiple modeling steps be written together and then run as a single unit.

Comparing two classifiers on the iris dataset

To make the idea of combining model outputs concrete, two classifiers were built and compared on the iris dataset.

Loading the data:

```
from sklearn.datasets import load_iris

data = load_iris()
X = data.data      # all the features
y = data.target    # the target labels
```

`data.feature_names` lists the feature columns, such as sepal length and petal length.

`data.target_names` lists the three target classes: setosa, versicolor, and virginica. Since this is a classification problem, checking the target names shows which classes the model needs to predict.

Splitting into training and test sets:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

`test_size` determines the percentage of the data kept aside as the test set (here, 20%).
`random_state` fixes the randomness of the split so the same split can be reproduced later.

Training two classifiers: logistic regression and a decision tree classifier were trained and compared. Both are classification models, since the task is to predict one of the three iris classes.

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

lr = LogisticRegression()
dtc = DecisionTreeClassifier(max_depth=3, random_state=42)

lr.fit(X_train, y_train)
dtc.fit(X_train, y_train)
```

Both models are fit using the training data only (`X_train` and `y_train`), never the test data.

Scoring the models:

```
score1 = lr.score(X_test, y_test)
score2 = dtc.score(X_test, y_test)

print(score1)
print(score2)
```

`.score()` evaluates a fitted model directly against test data (`X_test` and `y_test`) and returns its accuracy. Predictions can also be generated separately with `.predict()` and then scored, but calling `.score()` directly is a shortcut.

Watch out for: both models performed well in this exercise, but that does not mean one model always wins. Which model performs best depends on the dataset and use case — a different dataset built for the same comparison would be expected to favor a decision tree classifier instead. Re-using the same test data from one comparison to judge a different question would not produce a fair or conclusive result.

3. Key Takeaways

- Model selection is not about the single highest accuracy figure; the best model is the one that generalizes well to unseen data.
- Bias is error from oversimplifying the relationship in the data; variance is how much predictions change when the training data changes.
- A model too simple has high bias, low variance, and underfits. A model too complex has low bias, high variance, and overfits. The goal is a balanced model with moderate bias and variance.
- The bias-variance tradeoff is the search for the model complexity that minimizes overall error, since decreasing bias by adding complexity increases variance.
- Ensemble learning combines multiple models — averaging outputs for numerical data and using majority voting (mode) for categorical data — so that no single model's error dominates the final prediction.

Mental model: Think of the bias-variance tradeoff like choosing between health and convenience: improving one often costs something in the other, and the goal is finding the right balance where overall error is lowest.